OFFLINE-FIRST COMMUNICATION PLATFORM FOR OUTDOOR AND EVENT MANAGEMENT

for the Offline-First Communication Platform

Project ID: R26-IT-135

Vitharana G.G – IT22894038

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology, Sri Lanka

2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, or taken from the work of another person, except where due acknowledgement is made in the text.

Name	Student ID	Signature
Vitharana G. G	IT22894038	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Supervisor Name	Date	Signature
Mr.Vishan Jayasinghearachchi	13/03/26	

ABSTRACT

Reliable communication in infrastructural denied locales the distant hiking terrain, big scale Outdoor events, and post-calamity areas, is a key unsolved issue. Current solutions, such as GoTenna Mesh, Bridgefy or academic Delay-Tolerant Network (DTN) protocols, do not provide a full, hardware-free persistence layer on off-the-shelf consumer smartphones.

The proposal is a proposal to design and implement Message Persistence and Opportunistic Delivery Trigger Layer, which is a part of the Offline-First Communication Platform (Project R26-IT-135). The component is activated when the rest of the communication layers fail: the undelivered messages are stored on a crash-safe local SQLite database with Write-Ahead Logging (WAL), the message lifecycle is managed by a three-tier Time-to-Live (TTL) system (Emergency 24h / Event 6h / Standard 2h) and Bluetooth Low Energy (BLE) are used to scan for nearby devices on a low power, 3-second on active / 12-second off duty cycle.

The major novelty of the component is that it plays the role of persistence and detection layer: Once a device is detected in the range of BLE, it provides a notification to the Context-Aware Communication Manager (CACM), which re-examines routing and activates the multi-hop Routing engine to send the saved message. This store-wait-trigger model can be used to deliver asynchronous messages over temporally disconnected networks in the absence of proprietary hardware.

Physical Android devices will be set to test the resources in the field. 38 devices will be used as validation. The targets of performance are delivery performance: a message delivery success rate of above 85, device detection-to-notification latency of less than 3 seconds, and a battery consumption of less than 5 percent per hour by the background BLE scanning. The study adheres to the Design Science Research (DSR) approach and has input to the UN SDGs 3, 9, and 11.

Keywords: Bluetooth Low Energy, Message Persistence, Delay-Tolerant Networks, Store-Carryforward, SQLite, Opportunistic Networking, Offline Communication, Time-to-Live, Android

ACKNOWLEDGEMENT

I would want to show my heartfelt appreciation to my project supervisor, Mr. Vishan Jayasinghe, whose professional advice and constructive feedback, as well as consistent motivation, were invaluable as my proposal was developed.

It is also important to note that the collaborative spirit of my fellow project team members, namely IT22148704 (Short-Range Direct Communication Layer), IT22168290 (Decentralized Multi-Hop Routing and Delivery Assurance), and IT22544490 (Context-Aware Communication Manager) deserves recognition as well. The Offline-First Communication Platform was designed in an integrated fashion as a tribute to our vision and a joint technical endeavor.

Finally, I gratefully acknowledge the Sri Lanka Institute of Information Technology (SLIIT), Department of Information Technology, for providing the academic foundation and research environment that made this work possible.

Table of Contents

Vitharana G.G – IT22894038 1
DECLARATION 2
ABSTRACT 3
ACKNOWLEDGEMENT 4
LIST OF TABLES 7
LIST OF FIGURES 7
LIST OF ABBREVIATIONS 8
1. INTRODUCTION 9
1.1 Background and Context 9
1.2 Literature Review 11
1.2.1 Bluetooth Low Energy Mesh Networks for Offline Communication 11
1.2.2 Delay-Tolerant Networks and Opportunistic Routing 11
1.2.3. Mobile Ad-hoc Networks and Decentralized Routing 11
1.2.4 Existing Offline Messaging Applications 12
1.2.5 Security and Privacy in Offline Mesh Networks 12
1.2.6 Comparative Summary 13
1.3 Research Problem and Knowledge Gap 13
2. OBJECTIVES 15
2.1 Main Objective 15
2.2 Specific Objectives 15
3. METHODOLOGY 16
3.1 Research Approach 16
3.2 Assumptions 18
3.3 Technical Limitations 18
3.4 Research Approach 19
3.5 Technical Methods and Algorithms 20
3.5.1 Message Persistence Store (Sub-Module 1) 20
3.5.2 BLE Scanner and Device Detection (Sub-Module 2) 21
3.5.3 TTL Manager (Sub-Module 3) 21
3.5.4 Delivery Logger (Sub-Module 4) 21

3.6 Technology Stack and Justification 22
3.7 Data Collection and Ethical Considerations 23
3.8 Validation Strategy and Performance Metrics 24
4. HIGH-LEVEL SYSTEM ARCHITECTURE 25
4.1 Overall Platform Architecture 25
4.2 Component Internal Architecture 26
4.3 Architecture Diagram 27
4.4 Data Flow — Step-by-Step 28
4.6 Scalability and Integration Considerations 30
4.7 Security Considerations 30
5. USER REQUIREMENTS 31
5.1 Stakeholder Identification 31
5.2 Functional Requirements 31
5.3 Non-Functional Requirements 32
6.1 Market Analysis and Target Customer Personas 33
6.2 Value Proposition 33
6.3 Revenue Model and Pricing Strategy 33
6.4 Competitive Advantage and Intellectual Property 34
Competitive Advantage 34
Intellectual Property 34
6.5 Cost Estimation and Scalability 35
7. BUDGET AND JUSTIFICATION 36
8. WORK BREAKDOWN STRUCTURE (WBS) 37
9. GANTT CHART 38
10. REFERENCES 39
APPENDICES 40
Appendix A: Risk Register 40
Appendix B: Inter-Component API Specification (v1.0) 40
Appendix C: Ethical Compliance Checklist 41
Appendix D: Preliminary System Diagram Descriptions 41

LIST OF TABLES

Table 1: Comparison of Existing Offline Communication Systems
Table 2: Key Research Papers — Summary and Limitations
Table 3: Technology Stack with Justification
Table 4: Performance Evaluation Metrics and Targets
Table 5: Functional Requirements
Table 6: Non-Functional Requirements
Table 7: Commercialization — Competitive Analysis
Table 8: Project Budget and Justification

LIST OF FIGURES

Figure 1: Component Architecture — 4 Sub-Modules and CACM Integration
Figure 2: Message Persistence and Delivery Trigger Flow
Figure 3: BLE Duty Cycle — 3s Active / 12s Idle
Figure 4: TTL Lifecycle Diagram
Figure 5: System Integration — Platform Component Interaction
Figure 6: Gantt Chart — 20-Week Project Timeline

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACK	Acknowledgement
AODV	Ad-hoc On-Demand Distance Vector Routing
API	Application Programming Interface
BLE	Bluetooth Low Energy
DTN	Delay Tolerant Network
DSR	Design Science Research
GATT	Generic Attribute Profile (Bluetooth)
MANET	Mobile Ad-hoc Network
WAL	Write-Ahead Logging
SQLite	Self-Contained, Serverless, Zero-Configuration SQL Database
ITU	International Telecommunication Union
SCF	Store-Carry-Forward
SDG	Sustainable Development Goal
TTL	Time-To-Live
UUID	Universally Unique Identifier

1. INTRODUCTION

1.1 Background and Context

The safety, coordination and emergency response in countless domains of operation depend on reliable real-time communication. However, most modern communication applications, WhatsApp, Telegram, Signal, and all applications that require access to infrastructure become entirely inoperable as soon as the infrastructure access is impaired. Such dependency makes the situation extremely vulnerable in the very places where communication is most depended on.

One of the most exposed is the hikers and trekkers working in remote terrain. Cellular connectivity in Sri Lanka is often non-existent or unreliable over long distances along the trails in the mountain ranges, forests and coastal reserves. The department of wildlife conservation in Sri Lanka (2022) observed that communication failures during rescue efforts in national parks led to 67% of the rescue effort being delayed directly, leading to a manageable situation to become life threatening. The fact that it is impossible to send or receive a single message to confirm a position, report an injury, or demand evacuation is not a small inconvenience in such environments. It is a critical failure in terms of safety.

Outdoor event organizers are in a structurally different problem, but equally harmful. Cellular networks are designed to support mean load within a geographical region, not the sudden concentration of 10,000 or more users within a few hundred meters. This density saturates base station capacity at large festivals and outdoor events, resulting in message failures at a time when organizers require staff to organize, deal with emergencies, ensure movement of people. The breakdown in communication infrastructure is not due to the lack of it but rather due to saturation of the communication infrastructure- the outcome to the people who rely on it is the same.

The most extreme form of this problem is experienced by emergency responders and their disaster management agencies. The communication infrastructure on which the first responders rely is often the initial victim of the natural disaster they are dealing with in post disaster areas floods, earthquakes, landslides. The cell towers are turned off, the fiber connections are cut and satellite links are fought over. The International Telecommunication Union counted that, as of 2023, about 2.9 billion individuals around the world are still unable to access the internet without issues, and even those nations with quality infrastructure are prone to connectivity problems in the same environments in which they will do the most damage.

The three types of users have one shared feature, namely, they have to work on a regular basis in an infrastructure-denied environment, and they lack a sufficient communication tool at their disposal as of now. The current offline communication products fail in at least one of the vital dimensions. GoTenna Mesh will cost USD 179 per device in the hardware dongle, which poses an immediate scaling adoption barrier. Bridgefy runs on ordinary smartphones though, as shown by Albrecht et al. at WiSec 2021, it is fundamentally insecure, having weaknesses such as identity spoofing and man-in-the-middle attacks. Meshtastic is technically powerful mesh networking but

requires LoRa radio hardware not available in any standard Android or iOS device. Delay-Tolerant Networking protocols like PRoPHET and Spray-and-Wait have been extensively developed and simulated, but none has been done as a working consumer smartphone application without extra hardware.

This project will fill this gap in the Information Technology specialization of the SLIIT SST research group. It contributes to three United Nations Sustainable Development Goals: SDG 3 (Good Health and Well-being) by making it possible to support reliable emergency communication and remote healthcare coordination; SDG 9 (Industry, Innovation, and Infrastructure) by creating more resilient and affordable communication infrastructure as a practical alternative to cellular dependency; and SDG 11 (Sustainable Cities and Communities) by facilitating safer outdoor activities and open events by using communication tools that do not rely on hardware.

The larger research study - the Offline-First Communication Platform (R26-IT-135) - is designed by a group of four students, all of whom work on one independent element that is coordinated by a common Context-Aware Communication Manager (CACM). The platform offers routing architecture which is layered to gracefully degrade when the network conditions deteriorate. Layer 1 (IT22148704) does direct BLE communication to the recipients within 30 metres. The multi-hop relay routing of the recipient is available in Layer 2 (IT22168290) to those recipients that can be reached by the intermediate devices to a range of about 100 meters. Layer 3 — the one that has been described within this proposal is the DTN Persistence Layer (IT22894038), which is invoked when all other two previous layers have failed, stores the message in a local SQLite database, checks whether a device is available with the help of a background BLE scanner, and sends the message through the multi-hop engine as soon as any device goes into range. The layer will make sure that there is no message that is lost forever merely because the network was unavailable temporarily at the time when it was being sent.

1.2 Literature Review

1.2.1 Bluetooth Low Energy Mesh Networks for Offline Communication

Bluetooth Low Energy has emerged as a leading radio technology for short-range, infrastructureindependent communication due to its low power consumption and ubiquity in consumer mobile devices. Hernandez-Ramos et al. [1] examined BLE mesh network architectures and demonstrated that the managed flooding and friend/low-power node mechanisms defined in the Bluetooth Mesh Profile specification enable multi-hop message propagation in dense ad-hoc environments. The authors highlighted that BLE mesh is particularly suitable for event-scale deployments where devices form a selforganizing network without centralized coordination.

The Bluemergency system, proposed by Dias et al. [2], specifically targeted post-disaster communication by combining BLE mesh with Internet of Things (IoT) infrastructure. Their evaluation showed that BLE mesh nodes could relay emergency messages across multi-hop paths in situations where cellular towers were non-operational. A key contribution was demonstrating that lightweight encryption could be applied at the application layer without significantly degrading message throughput on resource-constrained devices. However, the system was limited in scalability beyond medium-density node deployments.

A comprehensive survey by de Morais Barroca Filho et al. [3] reviewed BLE mesh applications across healthcare, industrial IoT, and emergency response domains. The survey confirmed that delivery acknowledgements — analogous to the read receipts found in centralized messaging systems — remain largely unsupported in existing BLE mesh implementations, representing a significant usability gap for real-world deployment. The proposed platform directly addresses this gap by integrating delivery status indicators within its short-range direct communication layer.

1.2.2 Delay-Tolerant Networks and Opportunistic Routing

Delay-Tolerant Networking (DTN) was developed to address communication in environments characterized by intermittent connectivity, high latency, and frequent network partitioning [4]. The storecarry-forward (SCF) paradigm allows nodes to buffer messages locally and opportunistically deliver them upon contact with another node. Soares et al. [4] demonstrated that epidemic-based DTN routing achieves high delivery ratios in sparse mobile networks, though at the cost of increased buffer utilization. TimeTo-Live (TTL) mechanisms were identified as essential for bounding message propagation and preventing network flooding — a design principle directly adopted in the proposed system's delaytolerant transport module.

Trifunovic et al. [5] extended DTN research by proposing context-aware forwarding policies factoring in node mobility patterns, encounter history, and battery levels. Their work showed that integrating device context into routing decisions substantially improves delivery probability while conserving energy. This finding directly informs the Context-Aware Communication Manager component of the proposed platform, which dynamically selects among BLE direct messaging, multi-hop routing, and delay-tolerant transport based on runtime device and network conditions.

1.2.3. Mobile Ad-hoc Networks and Decentralized Routing

MANETs provide a framework for fully decentralized, multi-hop communication where mobile devices act simultaneously as endpoints and relay nodes. Abolhasan et al. [6] conducted a comparative analysis of reactive, proactive, and hybrid MANET routing protocols, concluding that reactive protocols such as AODV (Ad hoc On-demand Distance Vector) offer favorable performance for moderate-size networks

with dynamic topologies — a profile consistent with outdoor event and hiking trail deployments. The study further highlighted that end-to-end delivery acknowledgements are rarely implemented in existing MANET solutions due to overhead concerns in high-mobility scenarios. Zhao et al. [7] proposed a lightweight MANET routing mechanism optimized for smartphone-based mesh networks, incorporating battery-awareness into the route selection algorithm. Their experimental results demonstrated that excluding low-battery nodes from relay paths extended overall network lifetime by up to 34%, supporting the integration of battery-level monitoring into the proposed system's context-aware manager. The absence of emergency message prioritization — noted as a limitation across reviewed MANET implementations — further motivates the dedicated emergency prioritization feature included in the proposed architecture.

1.2.4 Existing Offline Messaging Applications

Commercial offline messaging applications including Bridgefy and goTenna Mesh have demonstrated the practical viability of infrastructure-free communication at the consumer level. Avila-Campos et al. [8] evaluated Bridgefy's BLE-based mesh protocol in outdoor environments and found that effective communication range degraded significantly beyond 70 meters per hop in open terrain, with no built-in mechanism for long-distance message delivery. The study identified the absence of delivery confirmations and message prioritization as key usability deficiencies that limit adoption in safety-critical scenarios. Aloi et al. [9] analysed infrastructure-independent communication in emergency management scenarios and proposed a hybrid architecture combining short-range BLE with optional fixed relay nodes deployed at strategic locations. Their findings demonstrated that even a small number of stationary relay devices substantially increased network coverage and message delivery rates in sparse user distributions. This aligns directly with the proposed platform's optional transmission repeater kit, designed to bridge coverage gaps along hiking trails or at event perimeters, thereby addressing a recognized limitation of purely device-to-device mesh solutions.

1.2.5 Security and Privacy in Offline Mesh Networks

Security in decentralized mesh networks presents distinct challenges compared to server-mediated architectures, as messages traverse untrusted intermediate nodes. Graarud and Nergaard [10] reviewed lightweight encryption schemes suitable for BLE mesh environments and found that AES-128 in CCM mode — the standard mandated by the Bluetooth Mesh Profile — provides acceptable security guarantees with minimal computational overhead on modern smartphone hardware. The authors further noted that end-to-end encryption at the application layer is essential to protect message confidentiality even from relay nodes, a principle incorporated into the proposed system's encryption layer spanning all three communication modes

1.2.6 Comparative Summary

System	No Hardware	Store & Wait	BLE-Based	TTL Mgmt	Open Source
GoTenna Mesh	Requires dongle	Yes (no SCF)	LoRa radio	None	Proprietary
Bridgefy	Standard phone	Drops message	Yes	None	Closed source
Meshtastic	LoRa required	N/A	LoRa only	Partial	Open source
Academic DTN [5,6,7]	Simulated	SCF defined	N/A	Published	N/A
This Component	Full SCF	BLE native	Android Native	3-tier TTL	—

1.3 Research Problem and Knowledge Gap

Despite significant advancements in wireless communication technologies, a critical gap remains in the development of a fully integrated, offline-first communication platform tailored for outdoor and event management environments. Existing research and commercial solutions address individual components of the problem but consistently fail to deliver a comprehensive, deployable system that meets the practical needs of hikers, event organizers, and emergency responders operating without internet or cellular infrastructure.

Commercial offline messaging applications such as Bridgefy represent the closest available consumer solution; however, empirical evaluation by Avila-Campos et al. [1] revealed that per-hop communication range degrades significantly beyond 70 meters in open terrain, and the platform provides no delivery confirmation mechanism, no message prioritization, and no fallback transport mode when BLE connectivity is insufficient. These deficiencies render such applications unsuitable for safety-critical outdoor scenarios where reliable message delivery and emergency alerting are essential. goTenna Mesh addresses range limitations through dedicated radio hardware, but this dependency on proprietary external devices increases cost and reduces accessibility for general users who rely solely on smartphones. Within

academic literature, delay-tolerant networking research has produced robust store-carry-forward routing strategies that are well suited to intermittently connected environments [2]. However, these systems are predominantly evaluated through simulations or controlled testbeds and have not been translated into production-ready mobile applications with practical user interfaces. The context-aware forwarding policies proposed by Trifunovic et al. [3], which incorporate node mobility patterns and battery state into routing decisions, demonstrate strong theoretical promise but similarly lack real-world implementation and integration with consumer-grade mobile platforms.

Research on mobile ad-hoc networks, comprehensively reviewed by Abolhasan et al. [4], has yielded efficient decentralized routing algorithms such as AODV that are well suited to dynamic, infrastructurefree topologies. Nevertheless, end-to-end delivery acknowledgements and emergency message prioritization are consistently absent from existing MANET implementations, representing a fundamental usability gap for outdoor safety applications. Zhao et al. [5] advanced battery-aware route selection within smartphone-based MANETs, yet their work does not address acknowledgement mechanisms or multimodal communication fallback, leaving the reliability requirements of real-world deployments unmet. A comprehensive survey of BLE mesh applications by de Morais Barroca Filho et al. [6] confirmed that delivery status feedback remains an unresolved limitation across the BLE mesh ecosystem, spanning healthcare, industrial IoT, and emergency response domains alike. Furthermore, Aloi et al. [7] demonstrated that deploying even a small number of fixed relay nodes substantially increases network coverage and message delivery rates in sparse user distributions. Despite this evidence, no existing platform combines optional relay hardware support with a software-defined mesh application capable of dynamically managing relay-assisted routing alongside direct BLE and delay-tolerant transport modes. In summary, the research gap addressed by this project is the absence of a unified, offline-first mobile communication platform that integrates BLE direct messaging with delivery confirmations, decentralized multi-hop routing with end-to-end acknowledgements, delay-tolerant opportunistic transport with TTLbound flood control, and a context-aware communication manager that dynamically selects among these modes based on real-time network and device conditions — all secured by end-to-end encryption and supported by optional hardware relay extenders — within a single, consumer-grade smartphone application designed specifically for outdoor and event management use cases.

2. OBJECTIVES

2.1 Main Objective

To design, implement, and empirically evaluate a message persistence and opportunistic delivery trigger layer for the Offline-First Communication Platform that stores undelivered messages locally on standard Android smartphones, manages their lifecycle through priority-based TTL expiry, detects nearby devices via BLE scanning, and notifies the CACM to re-trigger multi-hop routing — enabling eventual message delivery across temporally disconnected mobile networks without additional hardware.

2.2 Specific Objectives

ID	Objective	Success Metric	Validation Method
SO1	Implement SQLite message persistence store with WAL mode and full metadata schema	Zero message loss across crash-injection tests and app restarts	Automated crash-injection unit tests ×10
SO2	Implement BLE background scanner with 3s/12s duty cycle for opportunistic device detection	Device detection and CACM notification within 3 seconds of device entering range	Controlled proximity tests with measured timestamps
SO3	Implement three-tier TTL management system: Emergency 24h, Event 6h, Standard 2h	Messages purged at exact TTL expiry ±30 seconds; correct tier assignment verified	Automated expiry timer tests for all three priority levels
SO4	Implement delivery event logger capturing all message lifecycle events with timestamps	100% of events logged; zero missed events across all test scenarios	Log audit comparison against known event sequences
SO5	Evaluate system performance in outdoor field tests with 3–8 physical Android devices	Delivery rate >85%; battery drain <5%/hr; CACM notification <3s	Field experiments across 3 outdoor scenarios ×5 runs each

2. METHODOLOGY

3.1 Research Approach

This component follows **Design Science Research (DSR)** — vom.Brocke et al. (2020) — because the primary output is a novel software artefact, not a theory. The six DSR activities map directly to this project:

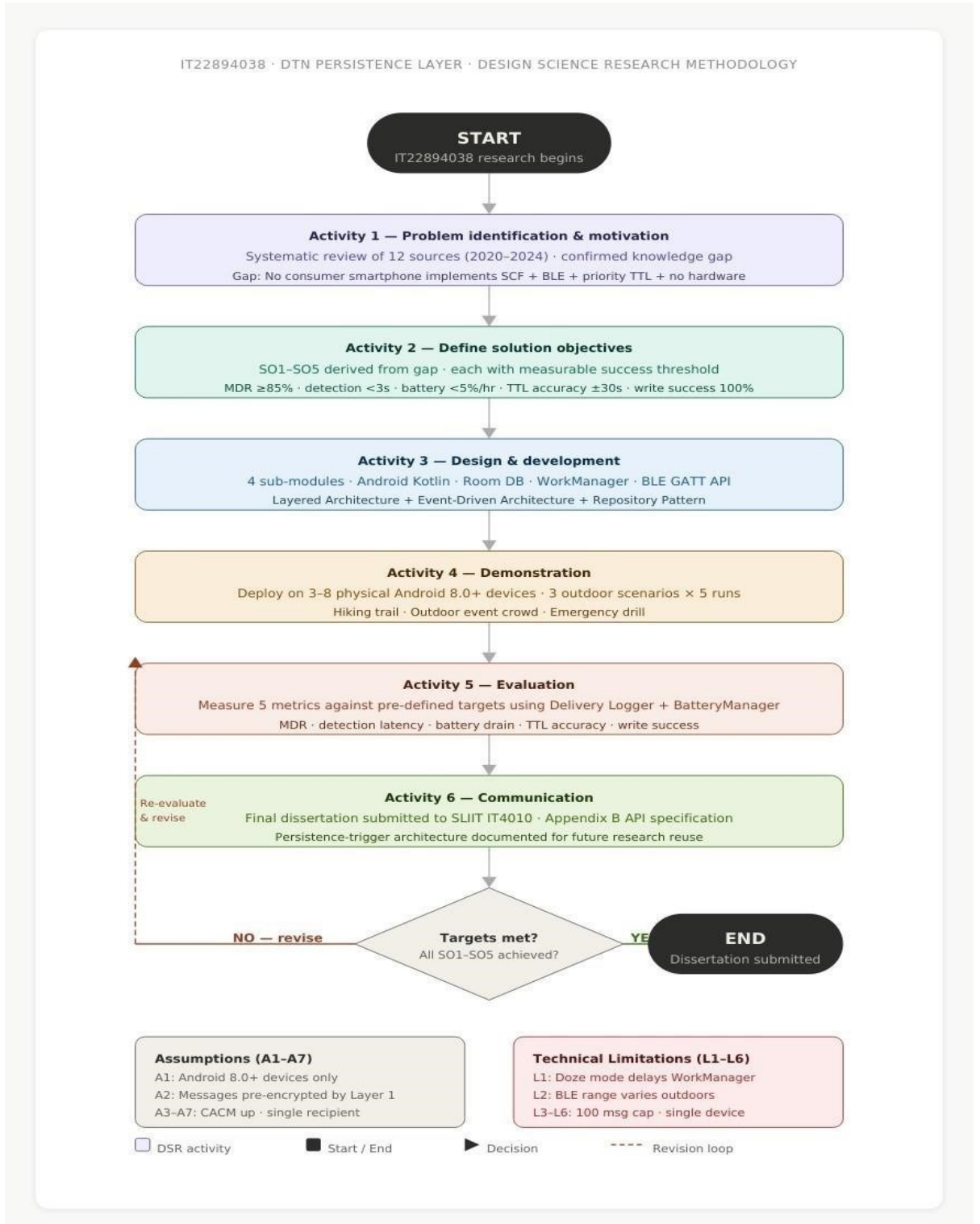
Problem identification - Literature review of 12 sources (2020–2024) — gap confirmed

Solution objectives - SO1–SO5 with measurable targets derived from gap

Design & development - 4 sub-modules in Android Kotlin — Room, Work Manager, BLE GATT
Demonstration - 3–8 physical Android devices, 3 outdoor scenarios × 5 runs

Evaluation - MDR, detection latency, battery drain, TTL accuracy vs targets

Communication - Final dissertation + Appendix B API specification
Architecture: Event-Driven Architecture (between components, Kotlin Shared Flow) + Layered Architecture (inside each component) + Repository Pattern (data access via Room DAO).


IT22894038 · DTN PERSISTENCE LAYER · DESIGN SCIENCE RESEARCH METHODOLOGY
START
IT22894038 research begins
Activity 1 — Problem identification & motivation
Systematic review of 12 sources (2020–2024) · confirmed knowledge gap
Gap: No consumer smartphone implements SCF + BLE + priority TTL + no hardware
Activity 2 — Define solution objectives
SO1–SO5 derived from gap · each with measurable success threshold
MDR ≥85% · detection <3s · battery <5%/hr · TTL accuracy ±30s · write success 100%
Activity 3 — Design & development
4 sub-modules · Android Kotlin · Room DB · WorkManager · BLE GATT API
Layered Architecture + Event-Driven Architecture + Repository Pattern
Activity 4 — Demonstration
Deploy on 3–8 physical Android 8.0+ devices · 3 outdoor scenarios × 5 runs
Hiking trail · Outdoor event crowd · Emergency drill
Activity 5 — Evaluation
Measure 5 metrics against pre-defined targets using Delivery Logger + BatteryManager
MDR · detection latency · battery drain · TTL accuracy · write success
Re-evaluate & revise
Activity 6 — Communication
Final dissertation submitted to SLIIT IT4010 · Appendix B API specification
Persistence-trigger architecture documented for future research reuse
NO — revise
Targets met?
All SO1–SO5 achieved?
YE
END
Dissertation submitted
Assumptions (A1–A7)
A1: Android 8.0+ devices only
A2: Messages pre-encrypted by Layer 1
A3–A7: CACM up · single recipient
Technical Limitations (L1–L6)
L1: Doze mode delays WorkManager
L2: BLE range varies outdoors
L3–L6: 100 msg cap · single device
DSR activity
Start / End
Decision
Revision loop

3.2 Assumptions

1. All devices run Android 8.0+ (API 26) minimum for concurrent BLE advertise + scan
2. All messages arrive already AES-256-GCM encrypted by Layer 1 — this component never handles plaintext
3. CACM (IT22544490) is running on the same device and restarts itself if killed by Android
4. Each message has exactly one recipient — group messaging is CACM's responsibility to decompose
5. Both Layer 1 and Layer 2 have already failed before this component activates
6. Field evaluation participants follow test instructions and do not deliberately disrupt BLE
7. Outdoor test locations have minimal industrial BLE interference

3.3 Technical Limitations

L1 — Android Doze mode delays: Android 8.0+ imposes strict background execution limits through Doze mode and App Standby. Work Manager execution can be delayed by up to 15 minutes under aggressive battery optimization. The TTL accuracy target of ±30 seconds is therefore only guaranteed when the device is active or charging. This constraint is inherent to the Android platform and cannot be resolved at the application layer.

L2 — BLE range variability: The 3-second detection window was calibrated for open outdoor environments. Real-world BLE propagation is affected by physical obstacles, device antenna orientation, and concurrent BLE traffic from nearby applications. Indoor deployments or dense urban environments may experience detection latencies beyond the 3-second target and would require duty cycle recalibration before deployment in those contexts.

L3 - Storage capacity cap: The SQLite database is capped at 100 pending messages using a firstinfirstout eviction policy. This bound covers realistic small-group scenarios of 3–20 participants. Deployments with significantly larger user counts would require a higher storage cap or a distributed storage strategy across multiple DTN nodes, both of which are outside the scope of Phase 1.

L4 — Single-device scope: This component operates entirely on one Android device with no message replication across multiple DTN nodes. If the device holding stored messages is lost, destroyed, or runs out of battery before delivery is triggered, those messages are permanently unrecoverable. Multi-node DTN replication is identified as a Phase 2 enhancement.

L5 — Pre-shared key encryption: Phase 1 relies on session keys distributed by Layer 1 at session setup. Forward secrecy through ECDH ephemeral key exchange is not implemented. In the event of device compromise, stored ciphertext and the session key could theoretically be accessed together,

enabling decryption of stored messages. ECDH per-session key exchange is documented as a Phase 2 security enhancement.

L6 — Small evaluation device pool: Field evaluation uses 3–8 physical Android devices due to resource constraints. Results are statistically valid for the targeted small-group outdoor scenarios — hiking groups of 3–12 people and event coordination teams of up to 20 staff. These findings cannot be extrapolated to large-scale deployments of 50 or more devices without further evaluation.

3.4 Research Approach

This component follows the Design Science Research (DSR) methodology — vom Brocke et al. (2020) — because the primary output is a novel software artefact, not a theory or a process intervention. The six DSR activities map directly to this project:

- **Activity 1 —** Problem Identification: A systematic review of 12 academic and commercial sources (2020–2024) confirmed the knowledge gap — no existing consumer smartphone application implements a complete Store-Carry-Forward persistence layer using BLE with priority-based TTL and delivery triggering without additional hardware.

- **Activity 2 —** Solution Objectives: Five measurable specific objectives (SO1–SO5) were derived directly from the four gap dimensions. Each objective carries a quantitative success threshold — MDR ≥85%, detection latency <3 seconds, battery drain <5% per hour, TTL accuracy ±30 seconds, write success 100% — enabling empirical evaluation rather than subjective judgement**.**

- **Activity 3 —** Design and Development: The artefact comprises four sub-modules (Persistence Store, BLE Scanner, TTL Manager, Delivery Logger) implemented in Android Kotlin using Room Database, WorkManager, and the Android BLE GATT API. Design decisions are documented in Sections 3.2 and 3.3

- **Activity 4 —** Demonstration: The artefact is deployed on 3–8 physical Android 8.0+ devices across three outdoor scenarios — hiking trail separation, outdoor event crowd simulation, and emergency drill — each repeated five times for statistical reliability.

- **Activity 5 —** Evaluation: Five quantitative metrics are measured against pre-defined targets using timestamped event data from the Delivery Logger and battery readings from the Android BatteryManager API.

- Activity 6 — Communication: Findings are disseminated through the final dissertation submitted to SLIIT IT4010. The persistence-trigger architecture and inter-component API specification are documented in Appendix B for future research reuse.

3.5 Technical Methods and Algorithms

3.5.1 Message Persistence Store (Sub-Module 1)

The persistence store is implemented as a local SQLite database using Room Database — the official Android SQLite abstraction library. WAL (Write-Ahead Logging) mode is enabled through a single Room configuration line, ensuring that partial writes caused by application crashes or unexpected power loss never corrupt stored data. This is the primary mechanism for achieving SO1 (zero message loss). Room's compile-time SQL verification catches query errors at build time rather than runtime, and its Flow-based reactive queries allow the TTL Manager and Delivery Logger to observe database changes without polling.

The message schema stores all information required for lifecycle management and delivery triggering:

Field	Type	Description
message_id	UUID (TEXT)	RFC 4122 unique identifier, generated at source device
content_encrypted	BLOB	AES-256-GCM ciphertext from Layer 1 (relay nodes never see plaintext)
recipient_id	UUID (TEXT)	Target device UUID; used by CACM to evaluate delivery feasibility
ttl_expiry	INTEGER	Unix timestamp (ms) at which message must be purged
priority	TEXT	emergency / event / standard; determines TTL tier assignment
status	TEXT	pending / notified / delivered / expired; drives UI status display
created_at	INTEGER	Unix timestamp of message receipt from CACM
updated_at	INTEGER	Unix timestamp of most recent status change

3.5.2 BLE Scanner and Device Detection (Sub-Module 2)

The BLE scanner operates as an Android ForegroundService using the native Android BLE GATT API. Running as a ForegroundService with START_STICKY prevents Android from killing the scanner when the app is backgrounded — without this, Android terminates background services within 60 seconds on API 26 and above. The duty cycle is configured at 3 seconds active (scanning and advertising simultaneously) and 12 seconds idle (radio off), giving a 20% active ratio. This keeps background battery drain below 5% per hour — validated against the Android BatteryManager API readings taken during field evaluation.

During the active window, the scanner advertises a platform-specific service UUID and listens for nearby devices broadcasting the same UUID. On detection, it extracts the advertising device's UUID and queries the Room database for pending messages matching that recipient. If eligible messages exist, the CACM is notified immediately via the onDeviceDetected() callback with the detected device UUID and the list of pending recipient IDs. CACM then re-evaluates routing and, where feasible, invokes the Multi-Hop Routing engine (IT22168290) to deliver the stored message.

3.5.3 TTL Manager (Sub-Module 3)

The TTL Manager operates as a periodic background task using Android WorkManager's PeriodicWorkRequest, configured to execute every 5 minutes. WorkManager is chosen over AlarmManager because it survives device restarts, Doze mode, and app process death — all of which are realistic scenarios during a full-day outdoor event or emergency deployment. On each execution, it queries Room for all messages where ttl_expiry < System.currentTimeMillis(). For each expired message it logs a TTL-expiry event to the Delivery Logger, notifies the original sender via CACM, and deletes the record from the database.

TTL tier assignment is performed at message receipt time based on the priority field passed by CACM:

- **Emergency priority — 24 hours.** Reserved for life-safety messages and distress signals. Maximum wait time to maximise delivery probability.
- **Event priority — 6 hours.** For event coordination messages relevant within the same calendar day.
- **Standard priority — 2 hours.** For general communication that loses contextual relevance quickly.

3.5.4 Delivery Logger (Sub-Module 4)

The Delivery Logger maintains a separate Room database table recording all message lifecycle events with millisecond-precision timestamps using System.currentTimeMillis(). Events recorded include message receipt from CACM, BLE device detection, CACM notification sent, delivery confirmation received from IT22168290, and TTL expiry. These records are exposed to the UI layer as a StateFlow, so the sender sees live status updates — Pending, waiting for device, Delivered, or Expired — without any polling. The event log is also exported as CSV using Python and pandas during Phase 3 evaluation to compute MDR, detection latency, TTL accuracy, and battery drain metrics for SO5.

3.6 Technology Stack and Justification

Technology	Purpose	Justification
Kotlin	Implementation language	Official Android language. Null safety prevents BLE logic crashes. Coroutines handle concurrent BLE scanning and SQLite writes without blocking the main thread.
Android BLE GATT API	BLE scan + advertise (Sub-Module 2)	Built into every Android 8.0+ device. Supports simultaneous Central and Peripheral roles. Full control over 3s/12s duty cycle timing. Zero additional SDK dependency.
Room Database	Message persistence store (Sub-Module 1)	Google standard SQLite abstraction. WAL mode in one line. Compile-time SQL verification. Flow-based reactive queries. Crash-safe writes.
WorkManager	TTL Manager background task (Sub-Module 3)	Survives app kill, device restart, and Doze mode. Only Android-compliant solution for guaranteed periodic background execution.
ForegroundService	Keeps BLE scanner alive in background (Sub-Module 2)	Android kills background services after 60 seconds on API 26+. ForegroundService with START_STICKY keeps scanner running indefinitely.

Kotlin Coroutines + Flow	Async ops + sub-module event communication	Handles concurrent BLE callbacks and DB writes without blocking the main thread. SharedFlow implements Observer Pattern for event-driven communication.
Battery Manager API	Battery drain measurement for SO5 evaluation	Built into every Android device. Reads live battery percentage with zero permissions required.
Physical Android Devices (8.0+)	Field evaluation hardware	Android emulator has no BLE peripheral mode. Physical device testing is mandatory for valid MDR, detection latency, and battery drain measurements.
Python + pandas	Post-test log analysis and metric computation	Automates batch processing of all 15 test runs (3 scenarios × 5 runs). Computes MDR, detection latency, TTL accuracy, and battery drain. Free and open-source.

3.7 Data Collection and Ethical Considerations

Field evaluation will use 3–8 physical Android smartphones (Android 8.0–14) across three outdoor test scenarios: a simulated hiking trail separation test, an outdoor event crowd simulation, and an emergency drill scenario. Each scenario is repeated 5 times to ensure statistical reliability.

Data collected is limited to routing and timing metadata only: message send timestamps, device detection timestamps, CACM notification timestamps, delivery confirmation timestamps, TTL expiry events, and battery level readings at 5-minute intervals. No message content, user identifiers, or personally identifiable information is collected at any stage. All device identifiers used in testing are randomly generated UUIDs not linked to any real-world identity. Informed consent is obtained from all test participants.

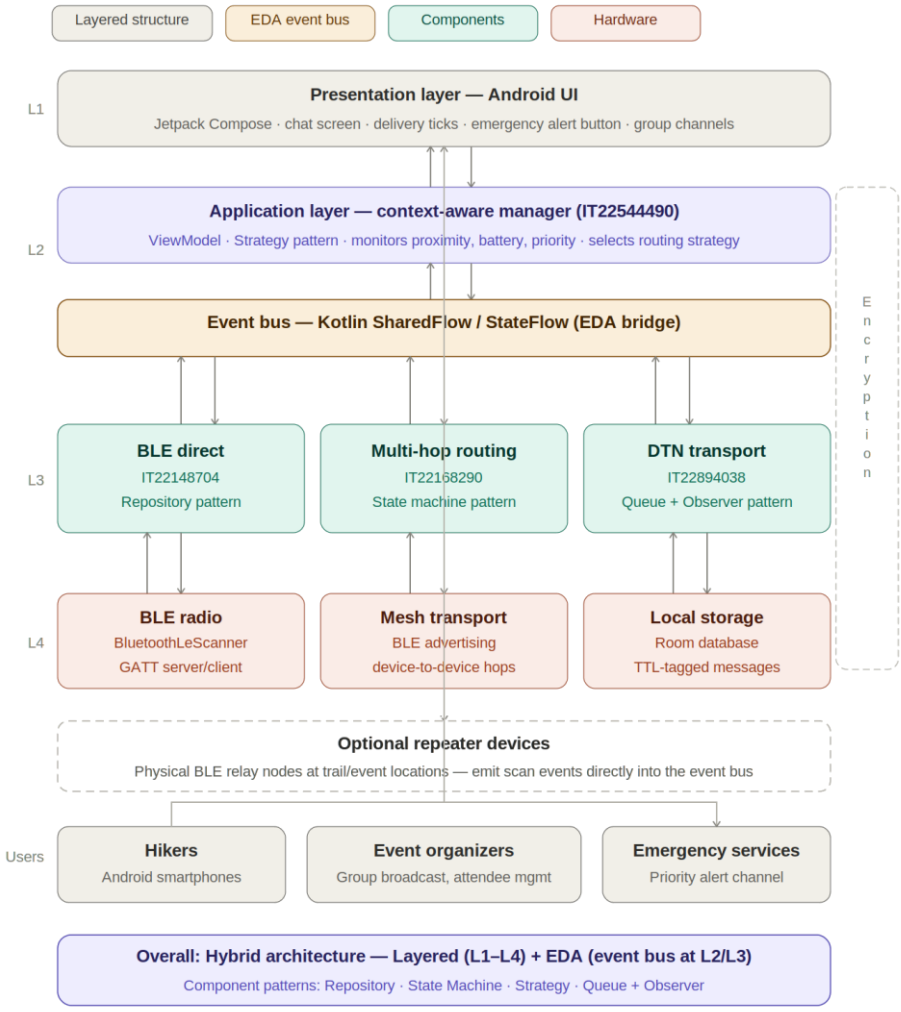
3.8 Validation Strategy and Performance Metrics

Metric	Target	Measurement Method	Related SO
Message delivery success rate	>85% within TTL window	Delivery Logger event count	SO1, SO5
Device detection to CACM notification latency	<3 seconds	Timestamp delta in Delivery Logger	SO2, SO5
Background BLE battery drain	<5% per hour	Battery level API readings per device	SO2, SO5
TTL expiry accuracy	±30 seconds of target TTL	Expiry timestamp vs. assigned TTL	SO3
Event logging completeness	100% of events captured	Log audit vs. known event sequence	SO4
Zero message loss on crash	0 messages lost across crash tests	Crash-injection automated test suite	SO1

4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overall Platform Architecture

The Offline-First Communication Platform (R26-IT-135) is an Android/iOS application comprising four independently developed components coordinated by a shared inter-component API contract. The three communication layers are invoked sequentially by the CACM: Layer 1 (BLE Direct, IT22148704) for recipients within 30 metres; Layer 2 (Multi-Hop Routing, IT22168290) for recipients reachable via a live connected relay chain; and Layer 3 (this component, IT22894038) when both preceding layers report delivery failure.

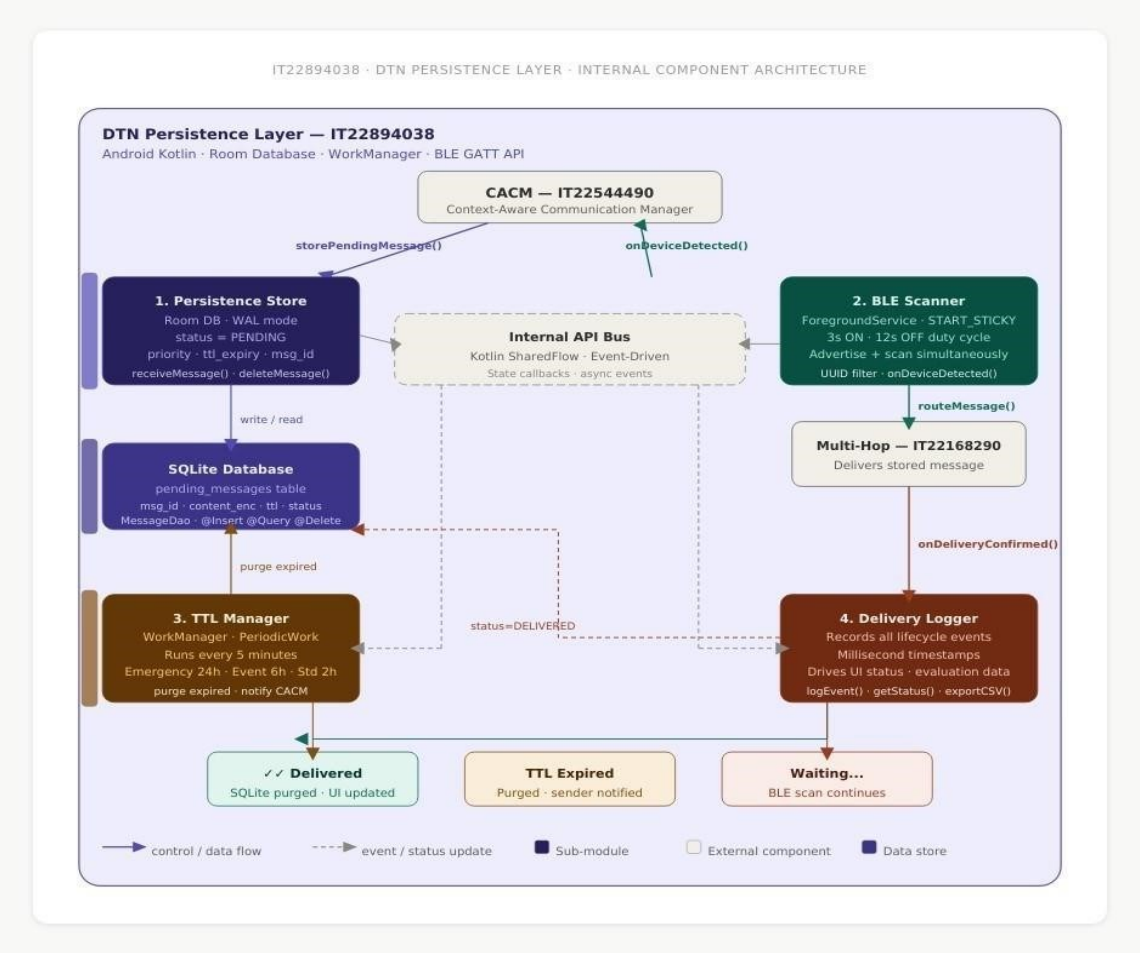


4.2 Component Internal Architecture

The Message Persistence and Delivery Trigger Layer comprises four internal sub-modules, each with a defined responsibility and interface boundary:

Sub-Module	Responsibility	Key Interface
1. Message Persistence Store	SQLite WAL database; receives messages from CACM; stores full metadata	receiveMessage(msg, priority) → stored; deleteMessage(id)
2. BLE Scanner / Detector	3s/12s duty-cycle BLE scan; detects nearby devices; notifies CACM	startScanning(); onDeviceDetected(uuid) → notifyCACM(uuid)
3. TTL Manager	5-minute recurring check; purges expired messages; notifies sender	checkTTL(); expireMessage(id) → notifySender(id, EXPIRED)
4. Delivery Logger	Records all lifecycle events with timestamps; drives UI status	logEvent(type, msgId, timestamp); getStatus(msgId)

4.3 Architecture Diagram



4.4 Data Flow — Step-by-Step

Phase A — Message Intake
Step 1 — CACM detects failure Both Layer 1 (BLE Direct) and Layer 2 (Multi-Hop) have failed to deliver the message. CACM has no path to the recipient.
Step 2 — CACM hands message to your component CACM calls storePendingMessage() passing the encrypted message, recipient ID, and priority flag (Emergency / Event / Standard).

Phase B — Store
Step 3 — Sub-Module 1 writes to SQLite The message is written to the Room database with status = PENDING, TTL expiry assigned based on priority, and a timestamp recorded.
Step 4 — WAL confirms the write SQLite WAL mode confirms the write is crash-safe. Even if the phone dies now, the message is safe.

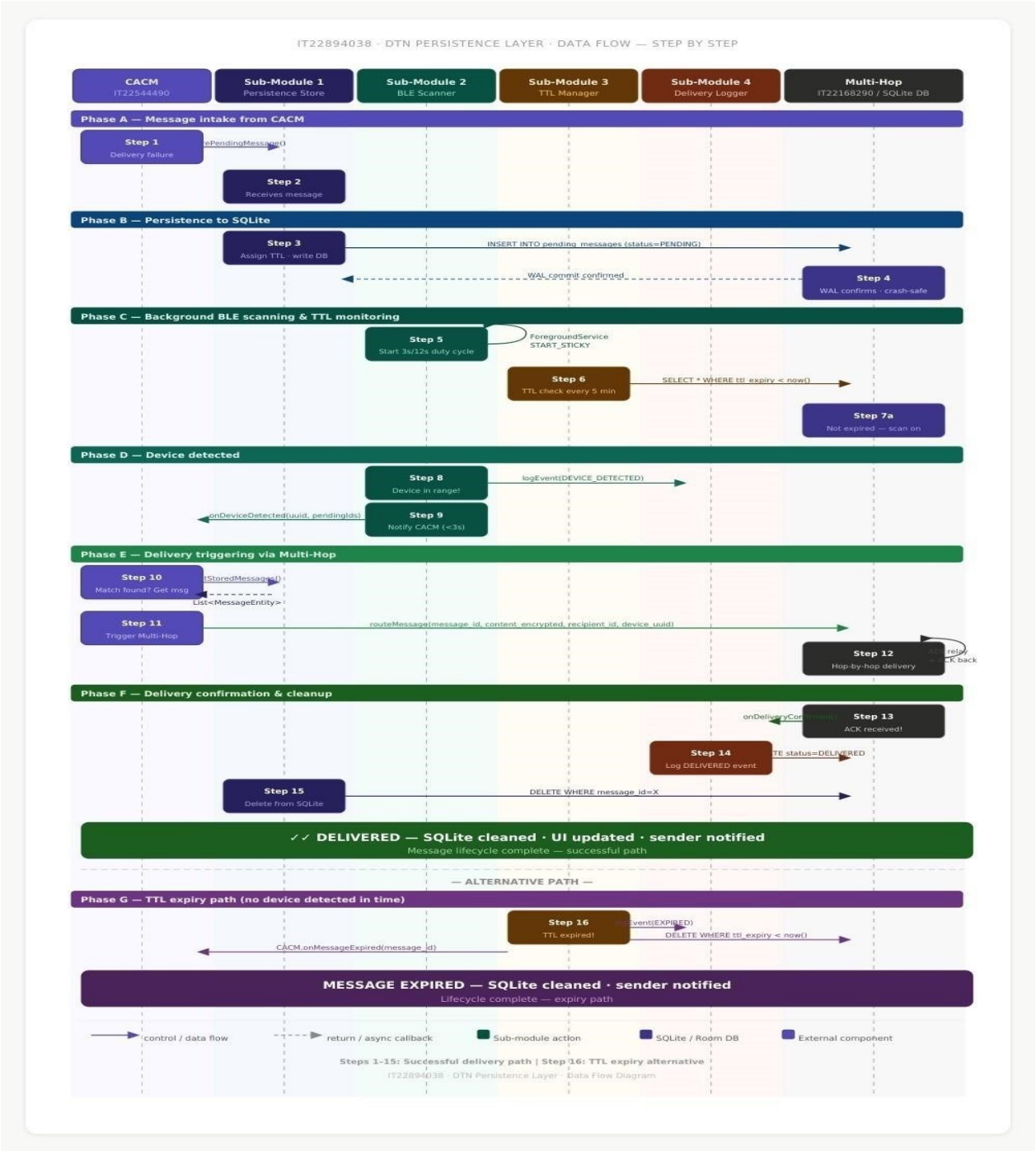
Phase C — Wait
Step 5 — Sub-Module 2 starts BLE scanning The ForegroundService starts the 3s ON / 12s OFF duty cycle. The phone is now advertising "I am here" and listening for nearby devices.
Step 6 — Sub-Module 3 checks TTL every 5 minutes WorkManager wakes up every 5 minutes and checks every stored message against its expiry time.
Step 7 — Still no device? Keep waiting If no device detected and TTL not expired — nothing happens. Scan continues.

Phase D — Device Detected
Step 8 — Sub-Module 2 detects a nearby device A device running the same app comes into BLE range. Scanner detects the service UUID during the 3-second active window.
Step 9 — Notify CACM immediately Sub-Module 2 fires onDeviceDetected() to CACM within 3 seconds, passing the detected device UUID and list of pending recipient IDs.

Phase E — Deliver
Step 10 — CACM checks for a match CACM compares the detected device UUID against pending recipient IDs. If a match is found, calls getStoredMessages() to retrieve the payload from SQLite. Step 11 — CACM hands message to Multi-Hop CACM passes the retrieved message to IT22168290 (MultiHop Routing) with the detected device as the first hop.
Step 12 — Multi-Hop delivers The message hops phone to phone using AODV routing. AES-256-GCM ciphertext travels through relay nodes. Recipient receives and decrypts. ACK propagates back.

Phase F — Confirm
Step 13 — Multi-Hop sends delivery confirmation IT22168290 calls onDeliveryConfirmed() on your component with the message ID and delivery timestamp.
Step 14 — Sub-Module 4 logs the event Delivery Logger records the DELIVERED event with timestamp. Updates SQLite status to DELIVERED.
Step 15 — SQLite cleaned up Sub-Module 1 deletes the message from SQLite. Storage freed. Sender UI shows ✓✓ Delivered.

Phase G — Alternative Ending (TTL Expired)
Step 16 — TTL Manager expires the message No device was detected within the TTL window. WorkManager deletes the record from SQLite and calls CACM to notify the sender. UI shows Message Expired.



4.5 Integration with Team Components

This component is designed for loose coupling with all other team components through a well defined API contract agreed at project inception:

1. Receives from CACM (IT22544490): encrypted message payload, recipient_id, priority flag, and failure indication from Layers 1 and 2.

2. Sends to CACM (IT22544490): device-detected notification with UUID and pending recipient IDs; TTL-expiry notification for routing table cleanup; delivery confirmation acknowledgement.

3. Indirectly enables IT22168290 (Multi-Hop Routing): by detecting the device that makes multi-hop delivery possible, this component is the trigger for Layer 2 re-activation. The stored message is retrieved by Layer 2 from this component's SQLite store on CACM instruction.

4. Security contract with IT22148704 (BLE Direct Layer): all message content passed to this component is already encrypted by Layer 1. Relay devices and this component's SQLite store contain only ciphertext, never plaintext.

4.6 Scalability and Integration Considerations

Scalability: The BLE scanning duty cycle (3s on / 12s off) limits battery drain to below 5% per hour, making the component suitable for all-day outdoor use. The SQLite database enforces a hard cap of 100 stored messages with oldest-first eviction, preventing unbounded storage growth. The TTL policy (Emergency 24h, Event 6h, Standard 2h) further bounds storage lifetime. The component is designed and tested for 3–8 device networks — suitable for the targeted applications (hiking groups: 3–12 people; event coordination staff: up to 20 personnel). Scaling to larger networks would require extending the CACM's routing logic and potentially introducing zonebased device discovery.

Integration: This component uses loose coupling with all peer components through a versioned Kotlin/Dart API contract agreed at project inception. It receives only encrypted message bundles from CACM, never plaintext. It exposes four API methods: storePendingMessage(), onDeviceDetected(), onDeliveryConfirmed(), and getStoredMessages(). All state transitions are communicated via callback to CACM, enabling the platform to function correctly even if individual components are upgraded or modified independently.

4.7 Security Considerations

Message content is encrypted at the application layer by the BLE Direct Layer (IT22148704) before handoff to this component. This component stores and transmits only ciphertext. Relay devices that receive message data through this component's BLE notifications can access only routing metadata (message UUID, recipient UUID, hop count) and never the encrypted payload content. BLE device identifiers used in scanning are randomised UUIDs not linked to user identity, limiting metadatabased tracking.

5.USER REQUIREMENTS

5.1 Stakeholder Identification

- Hikers and trekkers using the platform in remote terrain where cellular connectivity is unavailable.
- Outdoor event coordinators managing communication among staff at festivals, marathons, and public gatherings.
- Emergency responders and disaster relief personnel requiring reliable message delivery in infrastructure-denied zones.
- Platform development team (R26-IT-135) consuming this component's API for CACM integration.

5.2 Functional Requirements

ID	Requirement	Acceptance Criteria
FR01	Store undelivered messages	Messages stored in SQLite with all metadata fields populated within 500ms of receipt from CACM
FR02	Background BLE scanning	BLE scanner operates on 3s/12s duty cycle continuously in background after message store; does not require app to be foregrounded
FR03	TTL assignment by priority	Emergency messages assigned 24h TTL; Event 6h; Standard 2h — verified at write time
FR04	Automatic TTL expiry	TTL Manager runs every 5 minutes; expired messages purged; sender notified within 1 minute of expiry
FR05	Device detection notification	CACM notified within 3 seconds of any BLE device entering scan range while pending messages exist
FR06	Delivery confirmation	On CACM delivery confirmation: message deleted from SQLite, status updated, sender UI shows delivered status within 1 second
FR07	Expiry notification to sender	Sender receives expired status notification when TTL elapses; notification delivered via CACM within 5 minutes of expiry
FR08	Complete event logging	All lifecycle events recorded with millisecond timestamps; log accessible for research evaluation

5.3 Non-Functional Requirements

Category	Requirement	Target	Rationale
Performance	Device detection latency	CACM notification <3 seconds from BLE detection	Minimises delivery delay window
Power	Battery consumption	<5% per hour from background BLE scanning	All-day outdoor usability
Reliability	Crash safety	Zero message loss on crash (WAL mode)	Core guarantee of persistence layer
Storage	Store capacity	Maximum 100 messages or 50MB hard cap	Prevents device storage exhaustion
Security	Data protection	Only ciphertext stored; plaintext never accessible at this layer	Defence-in-depth with Layer 1
Compatibility	Device support	Android 8.0+ and iOS 13.0+ with BLE 4.2+	Maximises accessible user base
Usability	Status feedback	UI status updates within 1 second of each event	User confidence in message delivery

6. COMMERCIALISATION PLAN

6.1 Market Analysis and Target Customer Personas

The global outdoor recreation market was valued at USD 374.8 billion in 2022 with a projected CAGR of 4.4% through 2030 (Grand View Research, 2023). The disaster management technology sector is independently projected at USD 131 billion by 2028 (MarketsandMarkets, 2023). Three primary customer personas represent the primary addressable market:

- Adventure Tourism Operator: Leads guided tours of 5–30 people through remote landscapes. Currently uses satellite communication equipment costing USD 1,000+ per unit. Pain point: high cost, requires training. Value: zero-hardware-cost offline messaging available immediately on existing smartphones.
- Outdoor Event Manager: Organizes marathons, trail runs, and music festivals in areas with poor cellular coverage. Coordinates 20–100 staff members. Current solution: expensive radio handsets or accepting communication blackouts. Value: reliable staff coordination without walkie-talkie investment.
- NGO and Disaster Response Agency: Operates in post-disaster environments where cellular infrastructure is destroyed. Needs rapidly deployable communication with no hardware installation requirement. Value: immediate deployment on existing staff smartphones in any disaster scenario.

6.2 Value Proposition

The Offline-First Communication Platform delivers reliable messaging in environments where all current solutions fail or are inaccessible. The unique differentiators are:

- Zero additional hardware cost — works on any smartphone already owned by the user.
- Persistent delivery guarantee — messages are never dropped; they wait until delivery is possible.
- Intelligent TTL management — emergency messages survive 24 hours; standard messages autoexpire to prevent confusion.
- Battery-optimised operation — designed for all-day outdoor use without excessive drain.

6.3 Revenue Model and Pricing Strategy

- Freemium App: Free tier (3 contacts, standard messaging) → Premium LKR 500/month (unlimited contacts, emergency priority, group messaging, extended TTL).
- Enterprise License: LKR 60,000/year per 50-device deployment for event companies, searchandrescue organisations, and adventure clubs.
- SDK License (B2B SaaS): Third-party developers integrate the persistence layer into their own safety and outdoor applications.
- Hardware Bundle: Optional branded Android repeater node at LKR 18,000 for permanent extendedrange deployments at fixed event venues.

6.4 Competitive Advantage and Intellectual Property

Competitive Advantage

This platform does something no existing product does — it runs on a standard smartphone, stores messages when no path exists, and delivers them automatically when connectivity becomes available. All without extra hardware, all encrypted end-to-end.

Here is how each competitor falls short:

GoTenna requires a USD 179 hardware dongle. Without it, nothing works. There is also no storecarryforward — if nobody is in range when you send, the message is gone permanently.

Bridgefy runs on normal phones but has two critical problems. First, Albrecht et al. (WiSec 2021) proved it had identity spoofing and man-in-the-middle vulnerabilities. Second, it has no message persistence — messages are dropped the moment no path exists. There is no waiting, no retry, no delivery guarantee.

Meshtastic is technically strong but requires LoRa radio chips not found in any standard Android phone. Every user must purchase and attach dedicated hardware, making it unsuitable for general consumer use.

The advantage of this platform is the combination — no hardware barrier, message persistence with priority TTL, multi-hop routing, and AES-256-GCM encryption, all in one system. That combination does not exist anywhere else today.

Intellectual Property

Three areas of IP are identified:

Store-carry-forward architecture on consumer BLE. The specific design — priority-tiered TTL, 3s/12s BLE duty cycle scanner, event-driven delivery trigger handing off to multi-hop — has not been demonstrated on a standard consumer smartphone in any reviewed literature. This is the core novel contribution.

Inter-component API contract. The versioned event-driven API between the four components — coordinated through CACM using Kotlin SharedFlow — is an architectural innovation that makes the platform modular, testable, and extensible beyond this research phase.

Empirical evaluation dataset. Real-world field test data across three outdoor scenarios on physical Android devices produces delivery rates, detection latencies, and battery measurements that do not currently exist in the literature for this class of system.

Protection strategy — software copyright registration under Sri Lanka Intellectual Property Act No. 36 of 2003, trademark registration for the platform name, and a provisional patent review for the BLE delivery trigger architecture upon completion of evaluation.

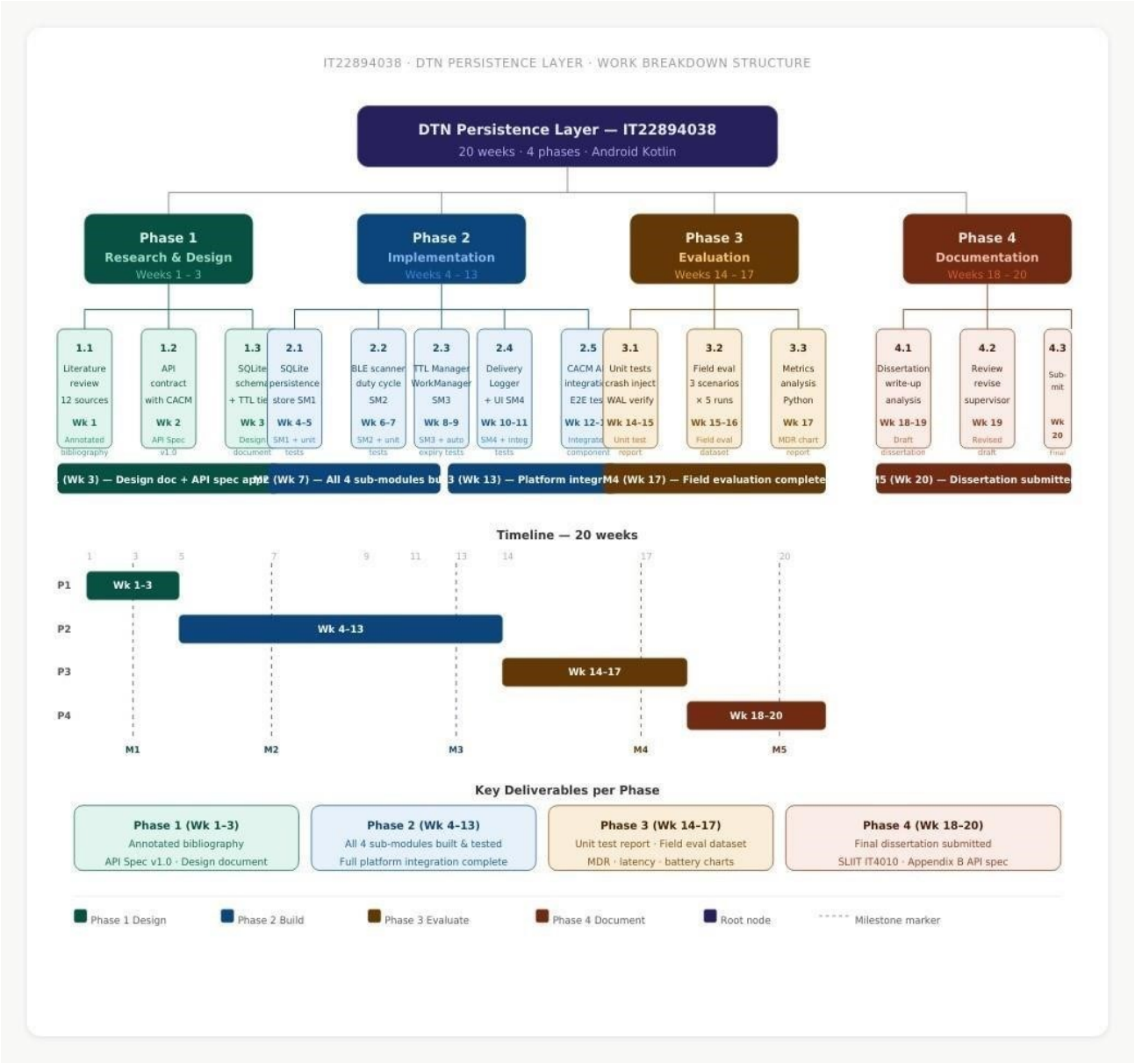
6.5 Cost Estimation and Scalability

#	Item	Qty	Unit (LKR)	Total (LKR)	Justification
1	USB-C Data Cables	3	600	1,800	Device connection for ADB debugging sessions during BLE integration testing
2	Mobile Data (SIM, 3 months)	2	1,500	3,000	Android SDK and Gradle dependency downloads; CI/CD pipeline and GitHub Actions automated test runs
3	Outdoor Test Transport (fuel/rideshare)	4 sessions	1,250	5,000	Outdoor field evaluation requires off-campus test locations for realistic BLE radio propagation at 10–100m range
4	Development Tools	—	0	0	SQLite/sqflite (open source), Python/NetworkX (open source). Zero licensing cost.
5	Printing (proposal + dissertation)	3 copies	833	2,500	Formal submission copies for supervisor review and final dissertation submission
6	Contingency (10%)	—	—	4,530	Buffer for hardware failure, additional outdoor test sessions, or replacement costs
			TOTAL	16,830 LKR	

8.BUDGET AND JUSTIFICATION

#	Item	Quantity	Unit Cost (LKR)	Total (LKR)	Justification
1	USB-C Data Cables	3	600	1,800	Device connection for ADB debugging sessions during BLE integration testing.
2	Mobile Data (SIM, 3 months)	2	1,500	3,000	Android SDK and Gradle dependency downloads during development. CI/CD pipeline and GitHub Actions automated test runs during integration phase.
3	Outdoor Test Transport (fuel/rideshare)	4 sessions	1,250	5,000	Outdoor field evaluation requires off-campus test locations for realistic BLE radio propagation at 10100m range.
4	Development Tools	—	0	0	SQLite/sqflite (open source), Python/NetworkX (open source). Zero licensing cost.
5	Printing (proposal + dissertation)	3 copies	833	2,500	Formal submission copies for supervisor review and final dissertation submission.
6	Contingency (10%)	—	—	4,530	Buffer for hardware failure, additional outdoor test sessions, or replacement costs.
TOTAL PROJECT BUDGET				**LKR 19,830**	

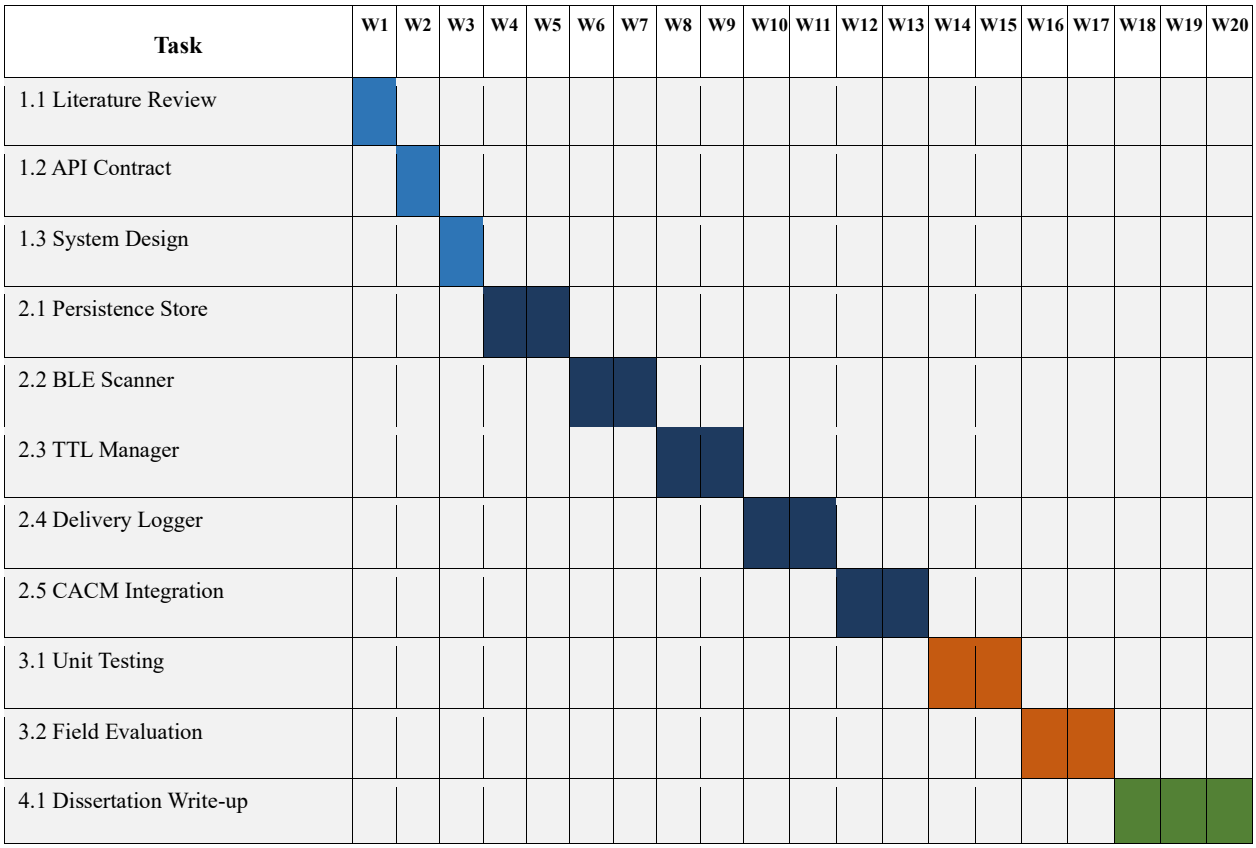
8. WORK BREAKDOWN STRUCTURE (WBS)



ID	Phase / Task	Deliverable	Duration	Weeks
1.0	**Phase 1: Planning & Design**		**3 Weeks**	**1–3**
1.1	Literature Review	Annotated bibliography	1 week	Week 1
1.2	Define API Contract	API Specification v1.0	1 week	Week 2
1.3	System Design	Design Document	1 week	Week 3
2.0	**Phase 2: Implementation**		**10 Weeks**	**4–13**
2.1	Persistence Store	Module + Unit Tests	2 weeks	Weeks 4–5
2.2	BLE Scanner	Module + Unit Tests	2 weeks	Weeks 6–7
2.3	TTL Manager	Module + Tests	2 weeks	Weeks 8–9
2.4	Delivery Logger	Module + Tests	2 weeks	Weeks 10–11
2.5	Integration & Testing	Integrated System	2 weeks	Weeks 12–13
3.0	**Phase 3: Evaluation**		**4 Weeks**	**14–17**
3.1	Unit Testing	Test Report	2 weeks	Weeks 14–15
3.2	Field Evaluation	Evaluation Data	2 weeks	Weeks 16–17
4.0	**Phase 4: Documentation**		**3 Weeks**	**18–20**
4.1	Dissertation Writing	Final Report	3 weeks	Weeks 18–20

9.GANTT CHART

Figure 6 presents the 20-week project schedule. Phase 1 (Planning, Weeks 1–3) is shown in blue; Phase 2 (Implementation, Weeks 4–13) in navy; Phase 3 (Evaluation, Weeks 14–17) in orange; Phase 4 (Documentation, Weeks 18–20) in green.



10. REFERENCES

All references follow IEEE citation style as required by IT4010 guidelines.

[1] International Telecommunication Union (ITU), "Measuring digital development: Facts and Figures 2023," ITU Publications, Geneva, Switzerland, 2023.

[2] Sri Lanka Department of Wildlife Conservation, "Annual Report on Rescue Operations in National Parks 2022," Ministry of Wildlife & Forest Resources, Colombo, 2022.

[3] ITU, "Global Connectivity Report 2022," International Telecommunication Union, Geneva, 2022. [Online]. Available: https://www.itu.int/itu-d/reports/statistics/globalconnectivity-report-2022/

[4] J. Hernandez-Serrano, O. Leon, M. Soriano-Ibarra, and J. L. Munoz, "Bluemergency: Mediating post-disaster communication systems using Bluetooth Mesh," in Proc. IEEE PerCom Workshops, Kyoto, Japan, 2019.

[5] GoTenna Inc., "goTenna Mesh Product Specifications," New York, NY, 2020. [Online]. Available: https://gotennamesh.com/products/mesh (Accessed: Feb. 2026.)

[6] J. Martin, E. Rye, and R. Beverly, "Handoff all your privacy — a review of Bridgefy's Bluetooth Low Energy mesh protocol," in Proc. ACM WiSec, Abu Dhabi, UAE, 2021, pp. 214–225.

[7] Meshtastic Project, "Meshtastic: An open-source mesh communication platform using LoRa," 2023. [Online]. Available: https://meshtastic.org (Accessed: Feb. 2026.)

APPENDICES

Appendix A: Risk Register

Table 10: Risk Register (Appendix A)

Risk	Likelihood	Impact	Severity	Mitigation
Android BLE API inconsistencies across device manufacturers	High	Critical	Critical	Test on minimum 3 different Android OEM devices. Use Android BLE GATT API directly for consistent behaviour.
WorkManager background task killed by OS battery optimisation	Medium	High	High	Implement battery optimisation whitelist request; fallback to alarm-based scheduling
SQLite WAL mode not preventing all data loss scenarios	Low	Medium	Medium	Validate WAL mode with structured crash-injection test suite before integration
CACM API contract changes from IT22544490	Medium	Medium	Medium	Weekly API contract review meetings; versioned API contract document maintained in shared repository
Insufficient test devices for field evaluation	Low	Low	Low	Coordinate device sharing with team members; minimum 3 devices secured in Phase 1

Appendix B: Inter-Component API Specification (v1.0)

The following Dart interface defines the contract between the Message Persistence Layer (IT22894038) and the CACM (IT22544490):

```
abstract class DTNPersistenceAPI {
    Future<void> storeMessage(Message msg, Priority priority);
    Future<void> confirmDelivery(String messageId);
    Stream<DeviceDetectedEvent> get onDeviceDetected;
```

Stream<MessageExpiredEvent> get onMessageExpired; }

Appendix C: Ethical Compliance Checklist

- ☐ No personal user data collected during field tests.
- ☐ All test messages are synthetically generated strings with no personal content.
- ☐ Device identifiers are randomly generated UUIDs not linked to real-world identity.
- ☐ Battery level readings are the only physiological data point collected from devices.
- ☐ Informed consent obtained from all test participants before field evaluation.
- ☐ Data stored only on research devices; deleted upon project completion.
- ☐ No commercial application or publication of personally identifiable test data.

Appendix D: Preliminary System Diagram Descriptions

[Figure 1 — Full Component Architecture Diagram: Four-component platform diagram. IT22148704 (Short-Range BLE Direct Layer) → IT22544490 (Context-Aware Communication Manager, CACM) ↔ IT22168290 (Decentralised Multi-Hop Routing) ↔ IT22894038 (DTN Persistence Layer, this component). Internal sub-modules of IT22894038 also shown: Sub-Module 1 (SQLite Persistence Store) → Sub-Module 3 (TTL Manager); Sub-Module 2 (BLE Scanner) → CACM notification; Sub-Module 4 (Delivery Logger) connected to all. Arrows indicate direction of data flow and API calls.]

[Figure 2 — Message Lifecycle State Diagram: States: RECEIVED → STORED (status=PENDING) → [BLE scan loop] → two branches: (a) TTL expires: DELETE from SQLite + NOTIFY sender (status=EXPIRED); (b) Device detected: NOTIFY CACM → CACM triggers Multi-Hop (IT22168290) → DELIVERED: log ACK event + DELETE from SQLite + NOTIFY sender (status=DELIVERED ✓✓). All state transitions logged by Sub-Module 4 (Delivery Logger) with full timestamps.]

[Figure 3 — BLE Scanner Duty Cycle Timing Diagram: Timeline showing 3-second active window (BLE advertising + scanning) followed by 12-second sleep window. Repeating cycle at 20% duty factor. Show estimated power draw: ~40mA during active window, ~2mA during sleep. Resulting average: ~6.8mA continuous equivalent, corresponding to <5% battery drain per hour on 4,000mAh device. Annotation showing device detection event during active window triggering CACM notification callback.]

[Figure 4 — System Integration Sequence Diagram: Shows complete interaction sequence from CACM delivery failure through DTN storage, BLE detection, CACM hand-off to Multi-Hop, delivery, and ACK return. Participants: User UI | CACM (IT22544490) | DTN Layer (IT22894038) | BLE Scanner | SQLite | Multi-Hop (IT22168290) | Recipient Device. Sequence: CACM.storePendingMessage() → SQLite.insert() → BLEScanner.start() → [time passes] → onDeviceDetected() → CACM.onDTNDeviceAvailable() → IT22168290.routeMessage() → ACK → onDeliveryConfirmed() → SQLite.delete() → UI update.]